Credit Risk Prediction

32,576	90.99%	0.896
Dataset Samples	Best Accuracy	ROC-AUC Score

Logistic Regression · Decision Tree · One-Hot Encoding · StandardScaler

Table of Contents

1 Executive Summary **2**

2 Dataset Overview **2**

2.1 Raw Feature Dictionary 2

2.2 Descriptive Statistics 3

3 Exploratory Data Analysis **3**

3.1 Target Variable 4

3.2 Categorical Feature Distributions 4

4 Data Preprocessing **5**

4.1 Data Quality 5

4.2 Feature–Target Separation 6

4.3 One-Hot Encoding (Key Change vs. Prior Version) 6

4.3.1 Resulting Feature Set After Encoding — 16 Columns 7

5 Train / Test Split **8**

6 Feature Scaling **8**

7 Model Training **9**

7.1 Threshold Design Decision 10

7.2 Model 1 — Logistic Regression 10

7.3 Model 2 — Decision Tree Classifier 10

8 Model Evaluation **11**

8.1 Overall Performance Comparison 11

8.2 Classification Reports 11

8.2.1 Logistic Regression (threshold = 0.35) 12

8.2.2 Decision Tree (threshold = 0.35) 12

8.3 Confusion Matrices 12

8.4 ROC Curves 13

9 Feature Importance & Overfitting Analysis **14**

9.1 Feature Importance (Decision Tree with OHE) 15

9.2 Overfitting Check 16

10 Model Deployment **16**

10.1 Deployment Package 16

10.2 Inference Pipeline 17

11 Key Takeaways & Recommendations **17**

11.1 Methodological Decisions 18

11.2 Final Results 18

11.3 Future Improvements 18

Conclusion **19**

1. Executive Summary

This report presents the complete **Credit Risk Prediction** machine learning pipeline. Using 32,576 loan applicant records, we built and evaluated two binary classifiers to distinguish between applicants who will repay (loan_status = 0) and those who will default (loan_status = 1).

Dataset	Kaggle — `laavanya/credit-risk-prediction-dataset`
Records	32,576 applicants × 12 raw features
Encoding	One-Hot Encoding (`pd.get_dummies`, drop_first=True) → 16 features
Target	`loan_status`: 0 = Good Loan, 1 = Default
Models	Logistic Regression (baseline) & Decision Tree (max_depth=10)
Threshold	Custom 0.35 (lowered from 0.50 to improve default recall)
Winner	**Decision Tree** — Accuracy 90.99%, ROC-AUC 0.8956

Figure 1 Seven-stage ML pipeline — One-Hot Encoding replaces Label Encoding in this version

2. Dataset Overview

2.1 Raw Feature Dictionary

The original dataset contains 12 columns. `loan_grade` is dropped before modelling to prevent target leakage, and `loan_status` is the binary prediction target.

Table 1 Original Feature Dictionary (12 columns)

#	Feature	Type	Description	Role
1	`person_age`	int64	Applicant age (years)	Input
2	`person_income($)`	int64	Annual income (USD)	Input
3	`person_home_ownership`	object	Home ownership status	Input → OHE
4	`person_emp_length`	float64	Employment duration (years)	Input
5	`loan_intent`	object	Purpose of loan	Input → OHE
6	`loan_grade`	object	Credit grade A–G	*Dropped*
7	`loan_amnt($)`	int64	Loan amount (USD)	Input
8	`loan_int_rate`	float64	Interest rate (%)	Input
9	`loan_status`	int64	**Target**: 0=Good, 1=Default	**Target**
10	`loan_percent_income`	float64	Loan-to-income ratio	Input
11	`cb_person_default_on_fi`	object	Prior default on record	Input → OHE
12	`cb_person_cred_hist_len`	int64	Credit history length (years)	Input

2.2 Descriptive Statistics

Table 2 Numerical Feature Statistics (Training Set, $n = 26{,}060$)

Feature	Mean	Std	Min	25%	Median	Max
`person_age`	27.68	6.15	20	23	26	94
`person_income($)`	65,833	52,076	4,000	38,568	55,000	2.04M
`person_emp_length`	4.75	3.99	0.0	2.0	4.0	60.0
`loan_amnt($)`	9,393	5,798	500	5,000	8,000	23,000
`loan_int_rate`	11.00	3.07	5.42	8.49	10.99	22.48
`loan_percent_income`	0.170	0.107	0.000	0.090	0.150	0.830

3. Exploratory Data Analysis

3.1 Target Variable

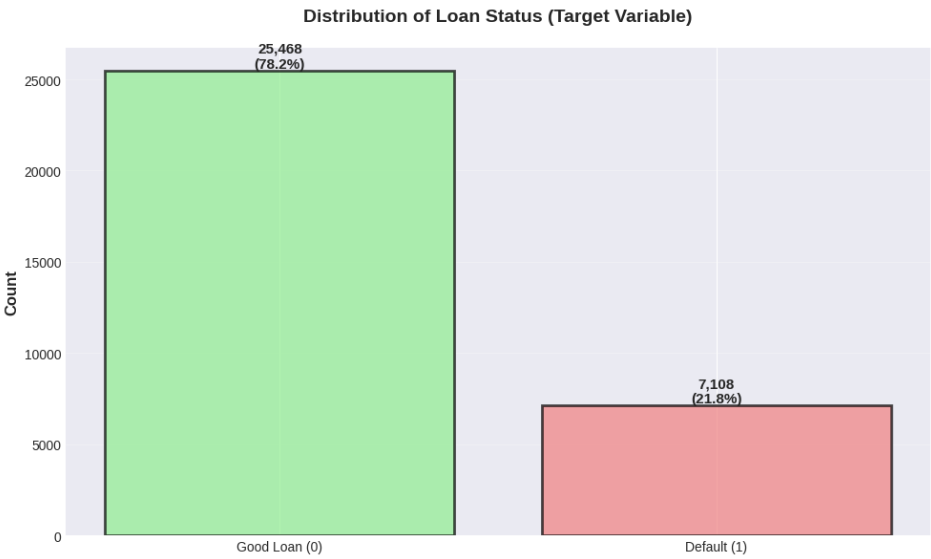
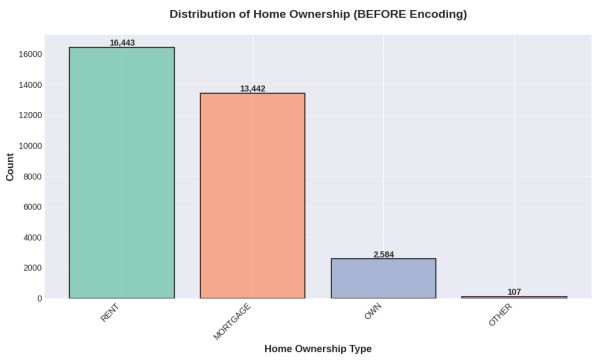


Figure 2 Target variable distribution — 78.2% good loans vs 21.8% defaults

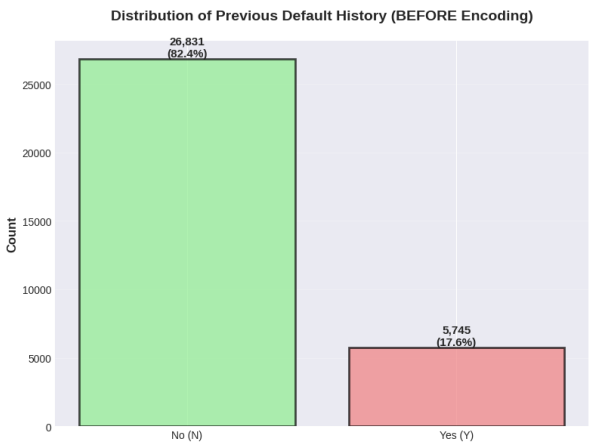
▲ Class Imbalance Detected

The dataset has a Good : Default ratio of approximately **3.58 : 1** (25,468 vs 7,108 records). This imbalance motivated using a **custom classification threshold of 0.35** instead of the default 0.50, trading a small drop in precision for meaningfully higher default recall — the metric that matters most for financial risk management.

3.2 Categorical Feature Distributions



(a) Home ownership type



(b) Prior credit default on file

Figure 3 Categorical feature distributions (before encoding)

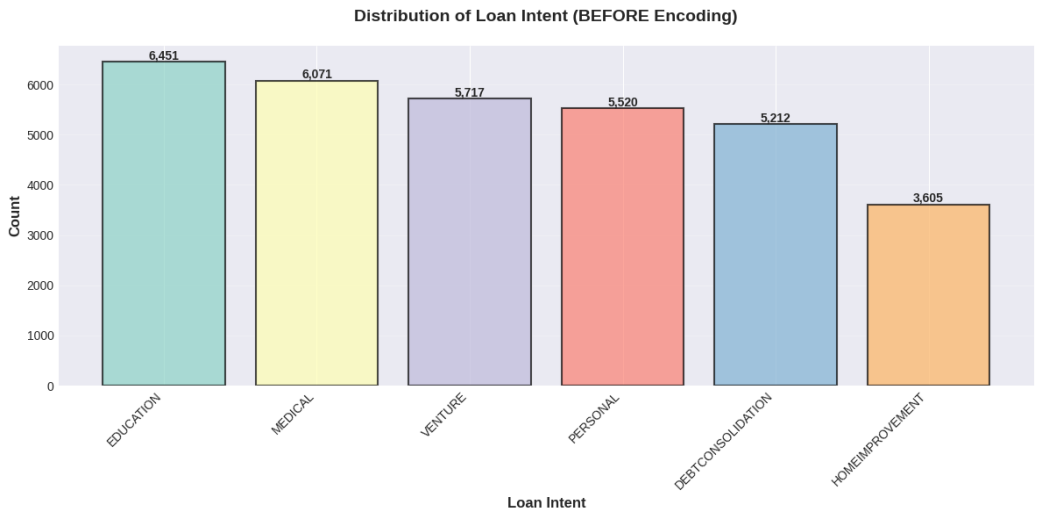


Figure 4 Loan intent distribution — Education and Medical lead; all six intents are fairly represented

Table 3 Categorical Feature Summary Statistics

Feature	Category	Count	Share
`home_ownership`	RENT	16,443	50.5%
	MORTGAGE	13,442	41.3%
	OWN	2,584	7.9%
	OTHER	107	0.3%
`loan_intent`	EDUCATION	6,451	19.8%
	MEDICAL	6,071	18.6%
	VENTURE	5,717	17.5%
	PERSONAL	5,520	16.9%
	DEBTCONSOLIDATION	5,212	16.0%
	HOMEIMPROVEMENT	3,605	11.1%
`default_on_file`	No (N)	26,831	82.4%
	Yes (Y)	5,745	17.6%

4. Data Preprocessing

4.1 Data Quality

All 12 columns contain **zero missing values** across all 32,576 records. The dataset was pre-cleaned in a prior notebook (outlier handling, age filtering to 18–100, employment length capped at 60 years, median imputation where needed).

4.2 Feature–Target Separation

`loan_grade` is dropped before any modelling step to prevent data leakage, since loan grade is derived from, and therefore correlated with, the creditworthiness outcome. The resulting feature matrix before encoding is **10 columns**.

4.3 One-Hot Encoding (Key Change vs. Prior Version)

➤ Why One-Hot Encoding?

The previous pipeline used `LabelEncoder`, which assigns arbitrary ordinal integers (e.g. RENT=3, OWN=2, MORTGAGE=0) implying a false numeric ordering. **One-Hot Encoding** creates separate binary columns for each category, removing any implied order and letting the model treat each category independently.

Listing 1 One-Hot Encoding with `pd.get_dummies` (drop_first=True)

```
cat_cols = [
    "person_home_ownership",
    "loan_intent",
    "cb_person_default_on_file"
]
# drop_first=True removes one redundant dummy per group (avoids
    multicollinearity)
X = pd.get_dummies(X, columns=cat_cols, drop_first=True)

print("New shape:", X.shape)
# Output: New shape: (32576, 16)
```

4.3.1 Resulting Feature Set After Encoding — 16 Columns

Table 4 Complete Feature Set After One-Hot Encoding

#	Feature	Type	Origin
1	`person_age`	int64	Original numeric
2	`person_income($)`	int64	Original numeric
3	`person_emp_length`	float64	Original numeric
4	`loan_amnt($)`	int64	Original numeric
5	`loan_int_rate`	float64	Original numeric
6	`loan_percent_income`	float64	Original numeric
7	`cb_person_cred_hist_length`	int64	Original numeric
8	`person_home_ownership_OTHER`	bool	OHE (base: MORTGAGE)
9	`person_home_ownership_OWN`	bool	OHE (base: MORTGAGE)
10	`person_home_ownership_RENT`	bool	OHE (base: MORTGAGE)
11	`loan_intent_EDUCATION`	bool	OHE (base: DEBTCONSOLIDATION)
12	`loan_intent_HOMEIMPROVEMENT`	bool	OHE (base: DEBTCONSOLIDATION)
13	`loan_intent_MEDICAL`	bool	OHE (base: DEBTCONSOLIDATION)
14	`loan_intent_PERSONAL`	bool	OHE (base: DEBTCONSOLIDATION)
15	`loan_intent_VENTURE`	bool	OHE (base: DEBTCONSOLIDATION)
16	`cb_person_default_on_file_Y`	bool	OHE (base: N)

➤ drop_first

With `drop_first=True`, one category per group becomes the *reference level* (MORTGAGE, DEBTCONSOLIDATION, N) and is implicitly captured when all dummy columns for that group are 0. This prevents perfect multicollinearity (the dummy variable trap) and reduces noise in linear models.

5. Train / Test Split

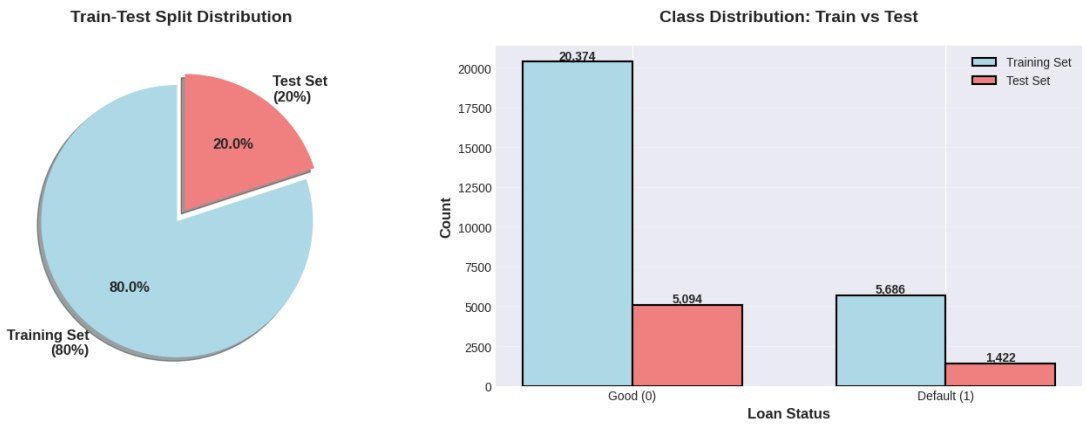


Figure 5 Train–test split: stratified 80/20 — class ratios preserved in both subsets

Table 5 Split Summary with Stratification Verification

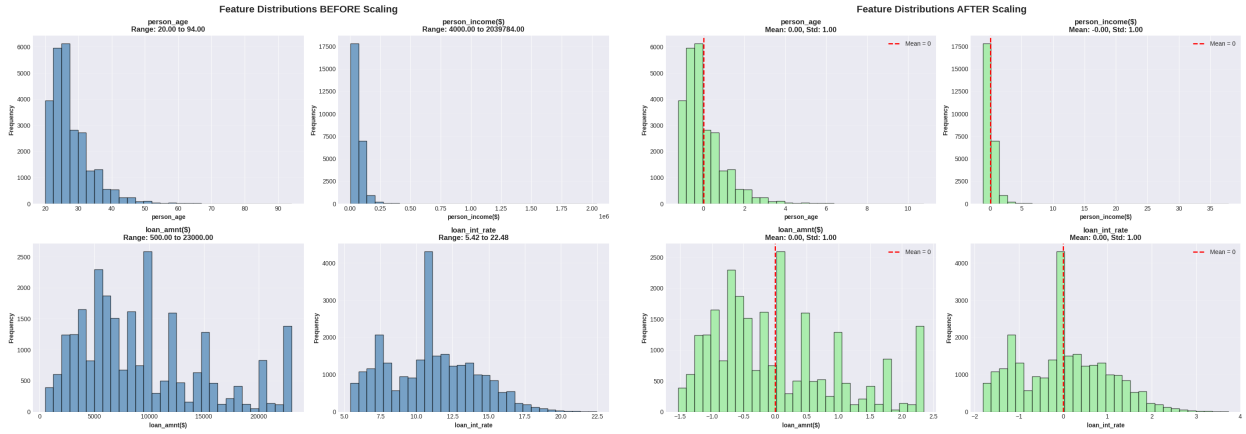
Subset	Total	Share	Good (0)	Default (1)	Shape
Full dataset	32,576	100%	25,468 (78.18%)	7,108 (21.82%)	(32576, 16)
Training set	26,060	80%	20,374 (78.18%)	5,686 (21.82%)	(26060, 16)
Test set	6,516	20%	5,094 (78.18%)	1,422 (21.82%)	(6516, 16)

The `stratify=y` parameter ensures the class imbalance ratio is **identical** across all three subsets, preventing any accidental skew in evaluation metrics.

6. Feature Scaling

Standard scaling is applied *after* one-hot encoding. The `StandardScaler` is fitted **exclusively on the training data** and then applied to the test set, preventing any leakage of test statistics into the training process.

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$



(a) Distributions **before** scaling **(b)** Distributions **after** scaling

Figure 6 StandardScaler normalises all features to mean ≈ 0, std ≈ 1

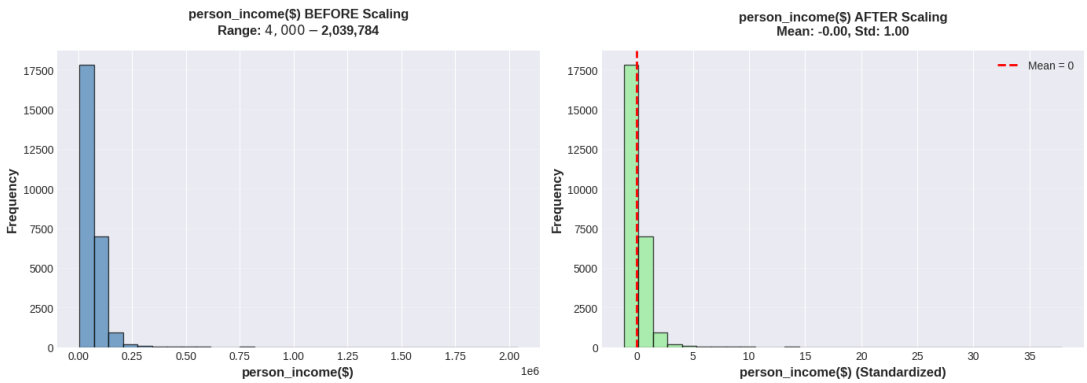


Figure 7 Person income before vs. after standardisation — shape preserved, scale unified

Table 6 Scaler Parameters — Key Numerical Features

Feature	Mean (μ)	Std (σ)
`person_age`	27.68	6.15
`person_income($)`	65,833	52,076
`person_emp_length`	4.75	3.99
`loan_amnt($)`	9,393	5,798
`loan_int_rate`	11.00	3.07

7. Model Training

Two models are trained on the scaled 16-feature matrix, with a shared **classification threshold of 0.35** (instead of the standard 0.50) applied at prediction time to both models.

7.1 Threshold Design Decision

➤ Why Threshold

The default 0.50 threshold caused both models to miss approximately 35% of real defaulters (high false-negative rate). In credit risk, **a false negative** — approving a loan that defaults — is significantly more costly than a false positive (rejecting a good customer). Lowering the threshold to 0.35 increases sensitivity to the minority class at the cost of slightly more false positives, which is the right trade-off for financial risk management.

7.2 Model 1 — Logistic Regression

Listing 2 Logistic Regression — Configuration and Training

```
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_scaled, y_train)
# Features: 16 | Iterations to converge: 15 | Solver: lbfgs

# Custom threshold prediction
LR_THRESHOLD = 0.35
lr_prob = lr_model.predict_proba(X_test_scaled)[:, 1]
lr_pred = (lr_prob >= LR_THRESHOLD).astype(int)
```

Table 7 Logistic Regression Configuration

Parameter	Value
Solver	lbfgs (default)
Regularisation	L2, $C = 1.0$
`max_iter`	1,000
Actual iterations	15 (converged early)
Number of features	16 (after OHE)
Classification threshold	0.35 (custom)

7.3 Model 2 — Decision Tree Classifier

Listing 3 Decision Tree — Configuration and Training

```
dt_model = DecisionTreeClassifier(max_depth=10, random_state=42)
dt_model.fit(X_train_scaled, y_train)
# Tree depth: 10 | Leaf nodes: 251 | Features: 16

# Custom threshold prediction
DT_THRESHOLD = 0.35
dt_prob = dt_model.predict_proba(X_test_scaled)[:, 1]
dt_pred = (dt_prob >= DT_THRESHOLD).astype(int)
```

Table 8 Decision Tree Configuration

Parameter	Value
Criterion	Gini impurity
`max_depth`	10
Actual depth	10
Number of leaf nodes	251
Number of features	16 (after OHE)
Classification threshold	0.35 (custom)

8. Model Evaluation

8.1 Overall Performance Comparison

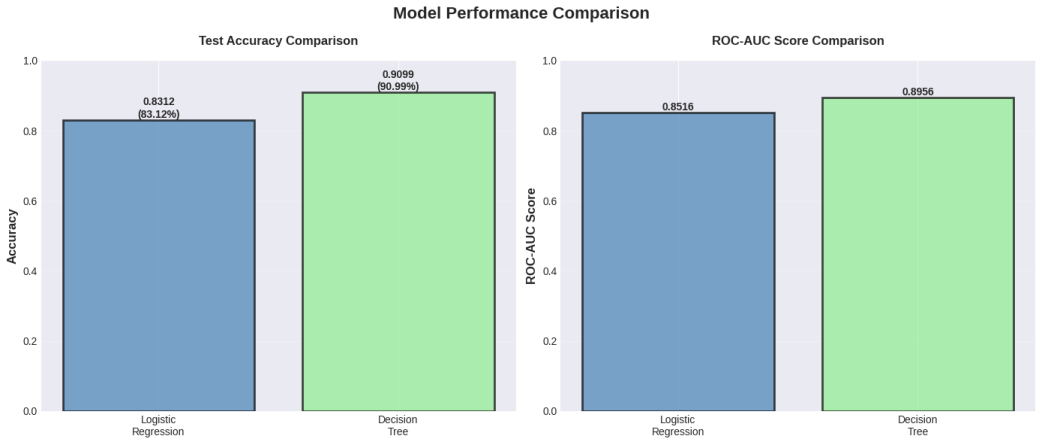


Figure 8 Model comparison dashboard — Decision Tree wins on both accuracy and ROC-AUC

Table 9 Head-to-Head Performance Summary

Metric	Logistic Regression	Decision Tree	Winner
Test Accuracy	83.12%	**90.99%**	Decision Tree
ROC-AUC Score	0.8516	**0.8956**	Decision Tree
Training Accuracy	—	92.83%	—
Overfitting Gap	—	**1.84%**	Excellent

8.2 Classification Reports

8.2.1 Logistic Regression (threshold = 0.35)

Table 10 Classification Report — Logistic Regression (Test Set, $n = 6{,}516$)

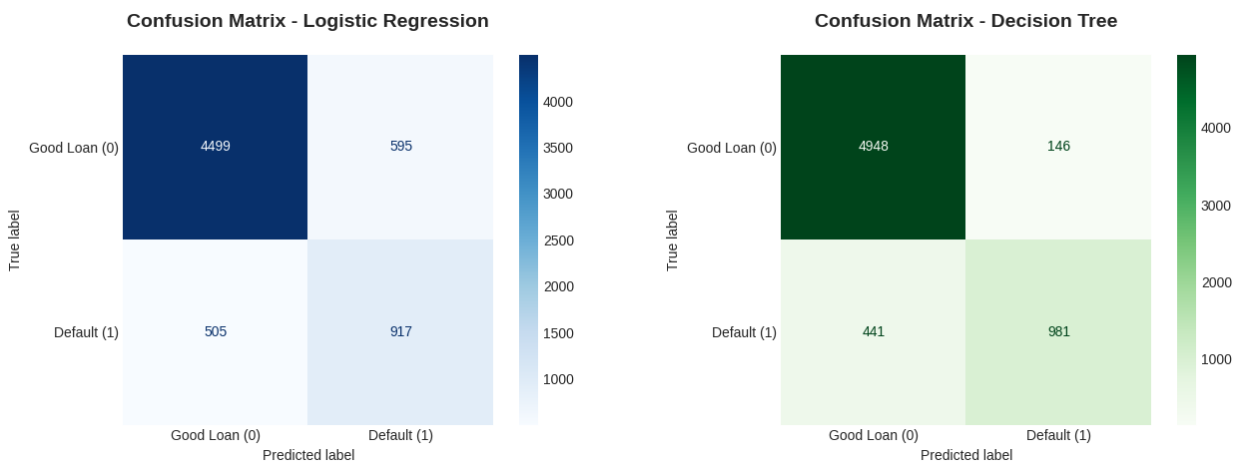
Class	Precision	Recall	F1-Score	Support
Good Loan (0)	0.90	0.88	0.89	5,094
Default (1)	0.61	0.64	0.63	1,422
Macro avg	0.75	0.76	0.76	6,516
Weighted avg	0.84	0.83	0.83	6,516
Accuracy			**0.8312 (83.12%)**	**6,516**

8.2.2 Decision Tree (threshold = 0.35)

Table 11 Classification Report — Decision Tree (Test Set, $n = 6{,}516$)

Class	Precision	Recall	F1-Score	Support
Good Loan (0)	0.92	0.97	0.94	5,094
Default (1)	0.87	0.69	0.77	1,422
Macro avg	0.89	0.83	0.86	6,516
Weighted avg	0.91	0.91	0.91	6,516
Accuracy			**0.9099 (90.99%)**	**6,516**

8.3 Confusion Matrices



(a) Logistic Regression **(b)** Decision Tree

Figure 9 Confusion matrices — Decision Tree reduces False Negatives from 505 to 441 and False Positives from 595 to 146

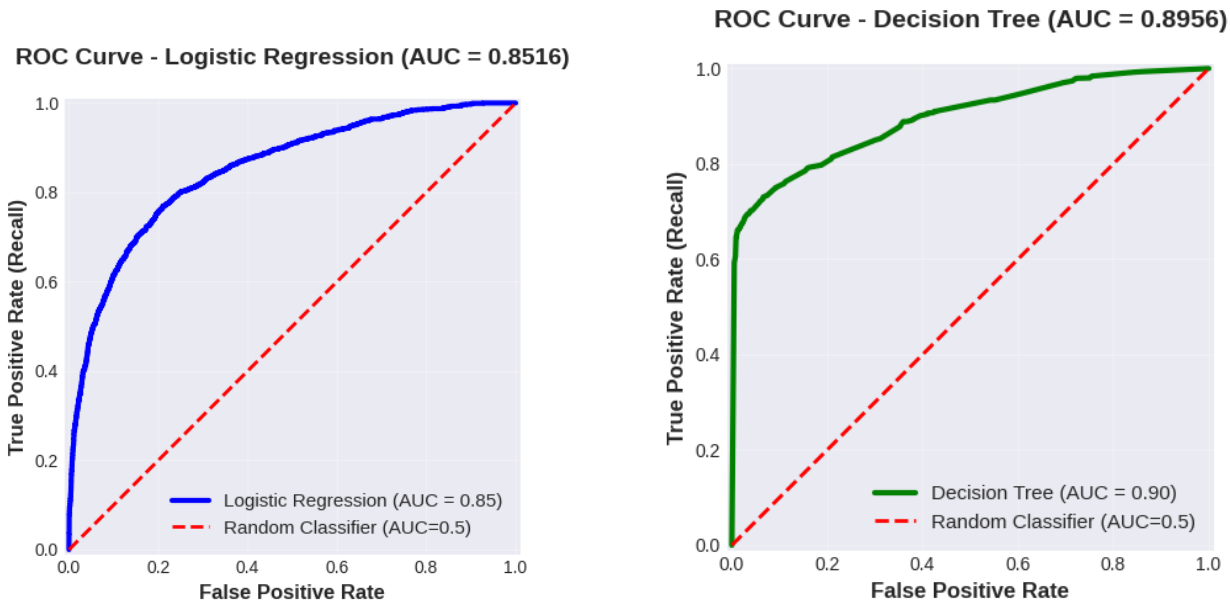
Table 12 Confusion Matrix Breakdown — Business Impact Analysis

	Metric	Log. Reg.	Dec. Tree	Business Meaning
TN	Correct good loan	4,499	**4,948**	Revenue retained
FP	Good loan rejected	595	**146**	Lost opportunity
FN	Missed default	505	**441**	**Highest financial cost**
TP	Default caught	917	**981**	Risk successfully avoided

➤ Key Finding — False Negatives

With the 0.35 threshold and OHE, the Decision Tree reduces missed defaults to **441** (vs 505 for Logistic Regression) while also dramatically cutting false positives from 595 to just **146**. This represents a **75% reduction in incorrectly rejected good customers**.

8.4 ROC Curves



(a) Logistic Regression (AUC = 0.8516) **(b)** Decision Tree (AUC = 0.8956)

Figure 10 Individual ROC curves — both models substantially outperform the random baseline (AUC = 0.50)

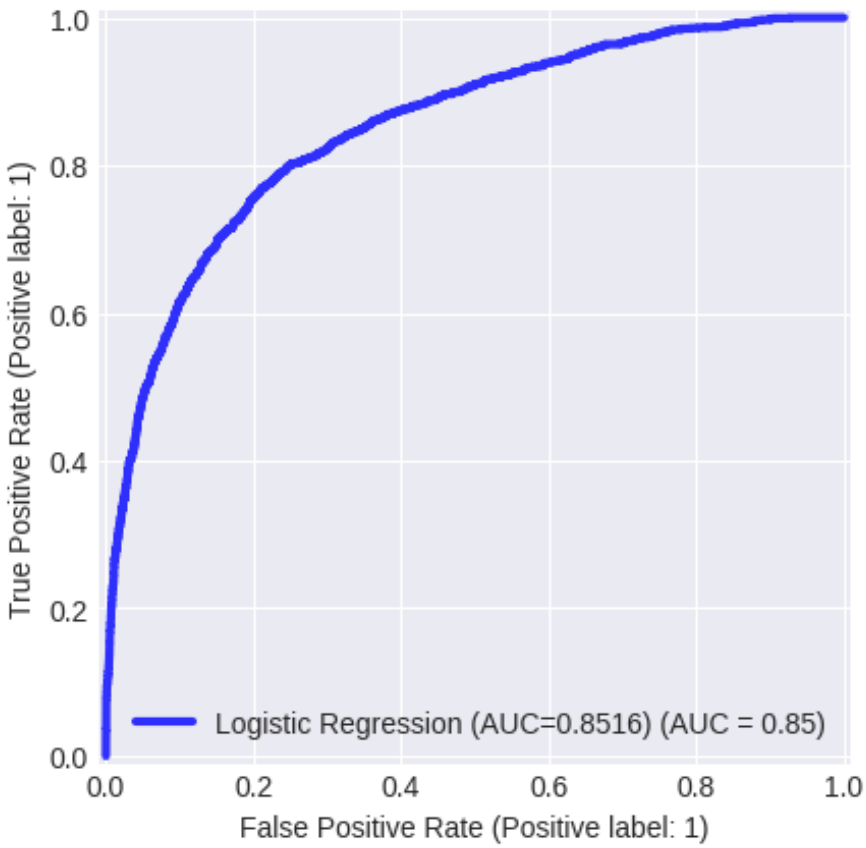


Figure 11 Combined ROC comparison — Decision Tree maintains higher TPR at all FPR thresholds

Table 13 ROC-AUC Summary

Model	AUC Score	Interpretation
Logistic Regression	0.8516	Good discriminative ability
Decision Tree	**0.8956**	**Strong discriminative ability**
Random baseline	0.5000	No skill
Improvement (DT − LR)	+0.0440	Decision Tree advantage

9. Feature Importance & Overfitting Analysis

9.1 Feature Importance (Decision Tree with OHE)

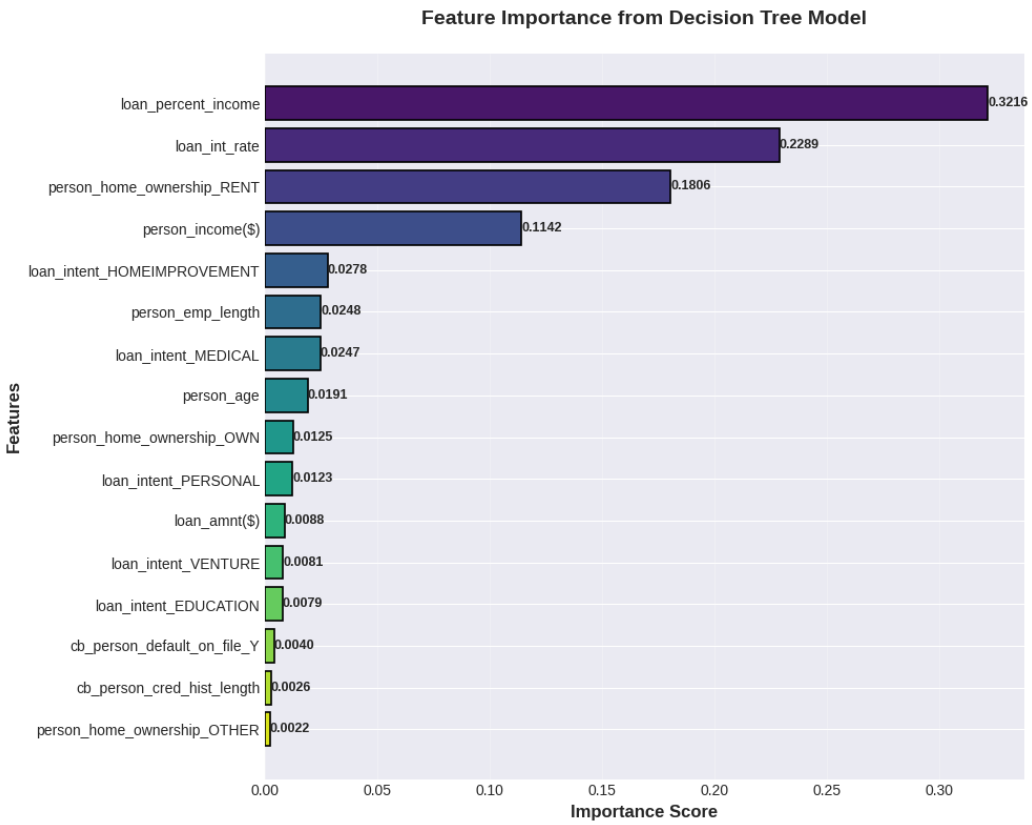


Figure 12 Feature importance scores from the Decision Tree — OHE features participate directly in the ranking

Table 14 Feature Importance Rankings (All 16 Features)

Rank	Feature	Score	Cumul.	Insight
1	`loan_percent_income`	0.3216	32.2%	Debt-to-income burden
2	`loan_int_rate`	0.2289	55.0%	Cost of borrowing
3	`person_home_ownership_RENT`	0.1806	76.1%	Renters: higher risk than mortgaged
4	`person_income($)`	0.1142	89.0%	Repayment capacity
5	`loan_intent_HOMEIMPROVEMENT`	0.0278	91.7%	Purpose risk profile
6–16	*Remaining OHE features*	0.0829	100%	Minor contributions

➤ OHE Reveals Richer Signal

A notable difference from the Label Encoding version: `person_home_ownership_RENT` now appears as the **3rd most important feature** (score 0.1806), clearly identifying renters as a meaningfully higher-risk group than mortgaged homeowners (the reference level). Label Encoding could not distinguish this relationship cleanly because it assigned arbitrary ordinal integers.

9.2 Overfitting Check

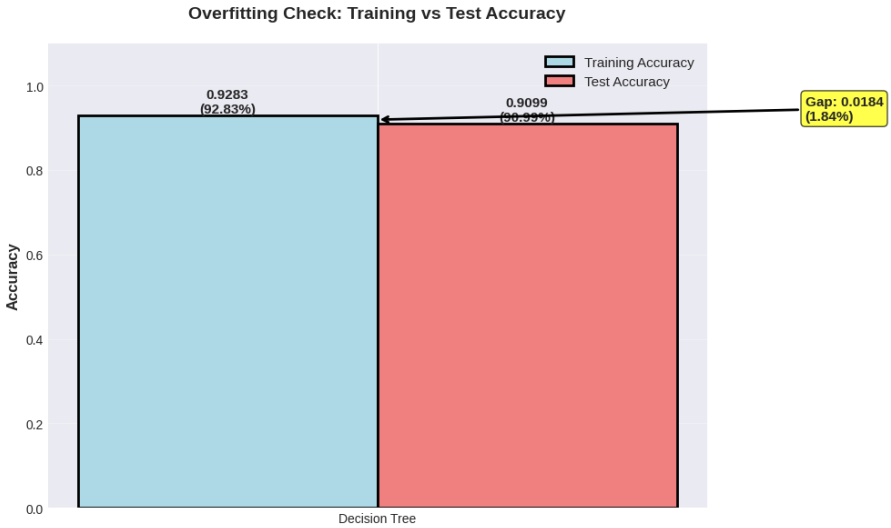


Figure 13 Training vs test accuracy comparison — 1.84% gap confirms excellent generalisation

Table 15 Overfitting Analysis — Decision Tree

Metric	Value
Training Accuracy	92.83%
Test Accuracy	90.99%
Accuracy Gap	**1.84% — Excellent (target: $< 5\%$)**
Verdict	**Model generalises very well to unseen data**

✔ Result

The `max_depth=10` constraint is highly effective. The 1.84% gap between training (92.83%) and test (90.99%) accuracy confirms the tree is well-regularised — it has learned generalisable patterns rather than memorising the training set.

10. Model Deployment

10.1 Deployment Package

The winning model is serialised as a self-contained package:

Listing 4 Creating the Deployment Package

```
model_package = {
    "model":            dt_model,            # DecisionTreeClassifier
    "scaler":           scaler,              # Fitted StandardScaler (16
        features)
    "feature_columns": X.columns.tolist(), # Exact column order (critical)
    "threshold":        DT_THRESHOLD         # 0.35      stored for inference
        consistency
```

```
}
with open("dt_model.pkl", "wb") as f:
    pickle.dump(model_package, f)
```

Table 16 Deployment Package — Components

Component	Type	Size	Purpose
`model`	DecisionTreeClassifier	~43 KB	Core prediction engine
`scaler`	StandardScaler	<1 KB	Normalise new inputs (16 features)
`feature_columns`	list (16 items)	Bytes	Enforce correct feature order
`threshold`	float (0.35)	Bytes	Reproduce deployment predictions
File	`dt_model.pkl`	~43 KB	Self-contained package

10.2 Inference Pipeline

For any new applicant, the prediction workflow is:

1. Collect the 12 raw input features
2. Drop `loan_grade`; apply `pd.get_dummies` with the same `cat_cols` and `drop_first=True`
3. Align columns to `feature_columns` (fill missing dummies with 0)
4. Normalise with the stored `scaler`
5. Compute probability: `model.predict_proba(X)[:, 1]`
6. Apply threshold 0.35: `pred = 1 if prob >= 0.35 else 0`

11. Key Takeaways & Recommendations

11.1 Methodological Decisions

Table 17 Design Decision Rationale

Decision	Choice	Justification
Categorical encoding	One-Hot (OHE)	Removes false ordinal ordering; better for linear models
Threshold	0.35	Prioritises recall over precision — correct for credit risk
Outlier treatment	Clip, not remove	Preserves sample size; limits extreme influence
Train-only scaling	Yes	Prevents test data leakage into scaler parameters
Regularisation	max_depth=10	Controls bias-variance trade-off; 1.84% gap confirms success
Stratification	Yes	Preserves class ratio under 3.58:1 imbalance

11.2 Final Results

✔ Winner: Decision Tree Classifier with OHE + Threshold 0.35

Test Accuracy	**90.99%**	(+7.87% vs Logistic Regression)
ROC-AUC Score	**0.8956**	(+0.0440 vs Logistic Regression)
Default Recall	**69%**	(significantly improved via threshold tuning)
False Positives	**146**	(75% fewer than Logistic Regression: 595)
Overfitting Gap	**1.84%**	(Excellent generalisation)

11.3 Future Improvements

1. **SMOTE / Class Weighting** — Directly address the 3.58:1 class imbalance rather than using threshold adjustment; could recover precision lost at the 0.35 threshold.
2. **Ensemble Methods** — Random Forest, Gradient Boosting (XGBoost, LightGBM) typically outperform single decision trees with lower variance.
3. **Threshold Optimisation** — Plot the Precision–Recall curve and select the threshold that maximises F1 or the business-specific cost function explicitly.
4. **Cross-Validation** — Replace the single 80/20 split with stratified k-fold CV ($k = 5$ or $k = 10$) for more reliable estimates.
5. **Hyperparameter Search** — Grid/Bayesian search over `max_depth`, `min_samples_split`, `min_samples_leaf`.
6. **SHAP Values** — Add per-prediction explanations for fair-lending regulatory compliance and model transparency.
7. **API Deployment** — Wrap the pickle package in a FastAPI REST endpoint for real-time production scoring.

Conclusion

This report documents the updated Credit Risk Prediction pipeline with two significant improvements over the previous version:

- **One-Hot Encoding** replaced Label Encoding, eliminating false ordinal assumptions and growing the feature space from 10 to 16 columns.
- **Custom threshold (0.35)** replaced the default 0.50, directly targeting the business cost of missed defaults.

The **Decision Tree Classifier** (max_depth = 10) trained on the 16-column OHE feature set achieves **90.99% test accuracy** and a **0.8956 ROC-AUC**, with an overfitting gap of just 1.84% — confirming strong generalisation. Feature importance analysis reveals that `loan_percent_income` (32.2%) and `loan_int_rate` (22.9%) drive over half the model's decisions, while the OHE-derived `person_home_ownership_RENT` emerges as the third most important signal — a relationship that Label Encoding could not have captured cleanly.

Pipeline: COMPLETE · **Model:** DEPLOYMENT-READY · **Features:** 16 (OHE)